COMP7940 Cloud Computing
Lab5: GitHub Actions with AWS EC2 Deployment

Intended Learning Outcomes

By the end of this lab, you should be able to:

- Understand GitHub Actions and its workflow components.
- Explain CI/CD (Continuous Integration / Continuous Delivery) in cloud deployment.
- Configure a GitHub Actions workflow for your chatbot project.
- Deploy automatically to AWS EC2 (Amazon Linux) whenever you push new commits.

Introduction to GitHub Actions and CI/CD

GitHub Actions is a CI/CD platform that automates build, test, and deployment pipelines.

- Continuous Integration (CI): Each commit triggers automated builds/tests to ensure smooth integration.
- Continuous Delivery (CD): Once validated, code is automatically deployed to a target environment (here, AWS EC2).

Workflow Components

- Workflow: Automated procedure stored in `.github/workflows`.
- Event: Trigger (e.g., commit push).
- Job: Collection of steps executed on a runner.
- Step: Individual task (action or command).
- Action: Reusable command block.
- Runner: Server executing jobs.

Preparing GitHub Secrets for EC2 Access

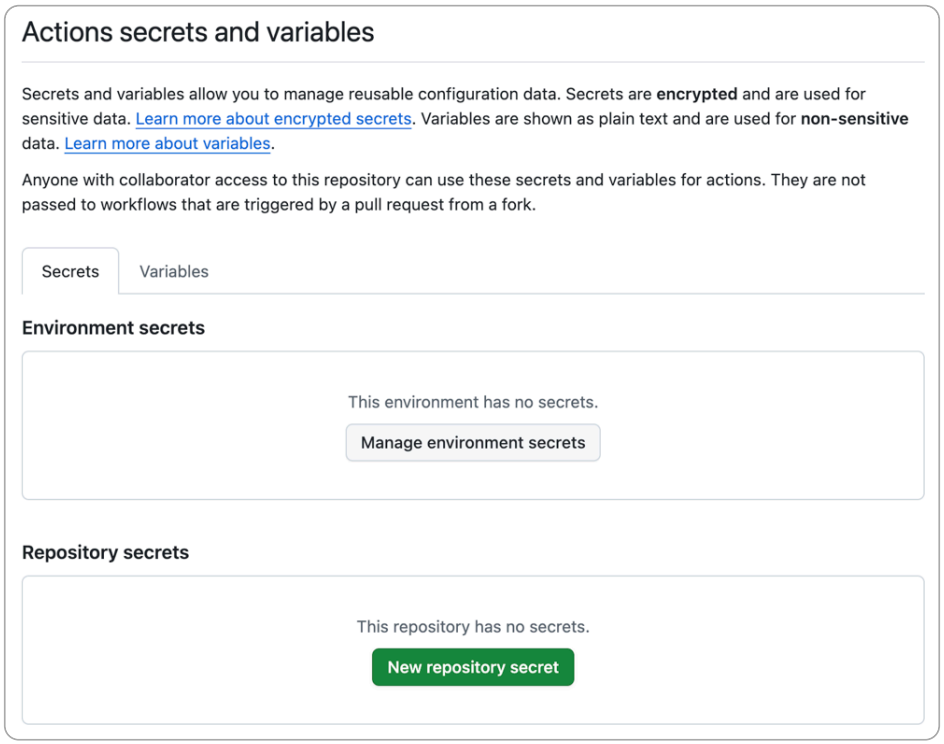
To allow GitHub Actions to connect securely to your AWS EC2 instance, you must configure **repository secrets** in GitHub. Secrets are encrypted values that can be referenced in workflows without exposing them in plain text.

Follow these steps:

1. Open your repository settings:
    - Go to your project repository on GitHub.

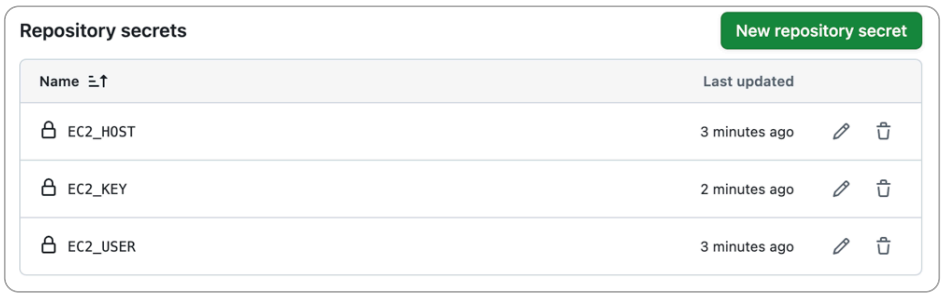
- Click on **Settings** in the top menu. (*not the settings of your profile*)

2. Navigate to Actions secrets:
   - In the left sidebar, select **Secrets and variables → Actions**.
   - This is where you will store sensitive information for GitHub Actions.



3. Click `New repository secret` to create three secrets with the following names and values:
   - `EC2_HOST`: The public IP address or DNS name of your EC2 instance.
   - `EC2_USER`: The default login user for Amazon Linux (usually `ec2-user`).
   - `EC2_KEY`: The private SSH key that matches the key pair you used when creating the EC2 instance. (paste the entire contents of the .pem file you downloaded when creating the EC2 key pair)

4. Verify the secrets :
   - After saving, you should see all three secrets listed under **Repository secrets**.
   - These values will be injected into your workflow file when GitHub Actions runs.



`EC2_HOST` `EC2_USER` `EC2_KEY`

Configure Workflow File

Now you will define the GitHub Actions workflow.

1. Create the directory locally:
   On your local computer inside your local repository, create the `.github/workflows` directory:

   ```
   mkdir -p .github/workflows
   ```

2. Create the workflow file:
   Inside that directory, create a file named `deploy.yml` with the following content:

   ```yaml
   name: AWS EC2 Deploy

   on:
     push:
       branches:
         - main

   jobs:
     deploy:
       name: Deploy to EC2
       runs-on: ubuntu-latest   # must be ubuntu-latest
       steps:
         - name: Checkout code
           uses: actions/checkout@v3

         - name: Setup SSH
           uses: webfactory/ssh-agent@v0.7.0
           with:
             ssh-private-key: ${{ secrets.EC2_KEY }}

         - name: Deploy application
           run: |
             ssh -o StrictHostKeyChecking=no ${{ secrets.EC2_USER }}@${{ secrets.EC2_HOST }} << 'EOF'
             cd ~/chatbot_comp7940
             git pull origin main
             # Restart chatbot inside venv
             pkill -f "python chatbot.py" || true
             nohup ~/chatbot_comp7940/venv/bin/python chatbot.py > app.log 2>&1 &
             EOF
   ```

Push and Deploy

Once your workflow file (`deploy.yml`) is created and saved in the `.github/workflows` directory of your local

repository, you need to push the changes to GitHub. This will trigger the GitHub Actions workflow and deploy your chatbot to the EC2 instance.

1. Check your repository status locally by running the following commands on your local computer inside the project directory:

   ```
   git status
   ```

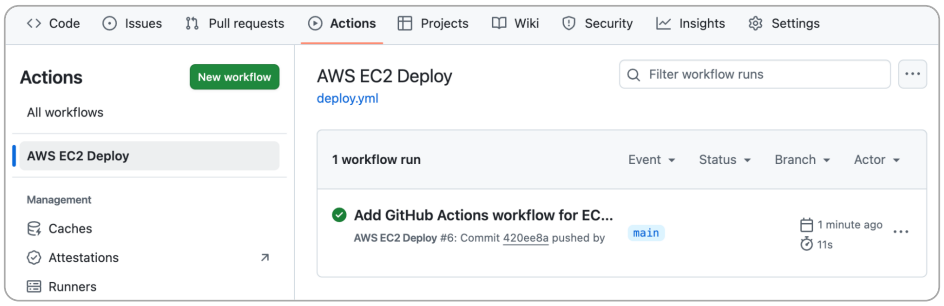
   This shows which files have been modified or added. You should see `.github/workflows/deploy.yml` listed.
2. Add the workflow file to your repository and commit:

   ```
   git add .github/workflows/deploy.yml
   git commit -m "Add GitHub Actions workflow for EC2 deployment"
   ```

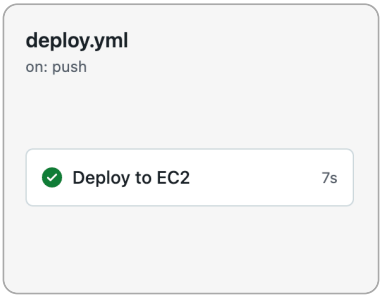
3. Push the commit to your remote repository:

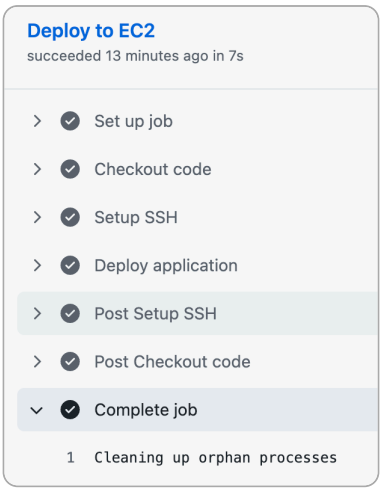
   ```
   git push origin main
   ```

4. Observe workflow run:
   - Once the push is complete, GitHub will automatically detect the workflow file and start running the job.
   - Navigate to your repository on GitHub.
   - Click on the **Actions** tab.
   - You should see a new workflow run named **AWS EC2 Deploy**.



5. Monitor the workflow run:
   - Click into the workflow run to see detailed logs.





Error Recovery

If your workflow run fails (red cross mark in GitHub Actions), follow this checklist:

1. Confirm EC2 instance status:
   - Ensure the instance is running and reachable via its public IP/DNS.

     > Note: The public IP/DNS of the instance may be changed after system reboot.

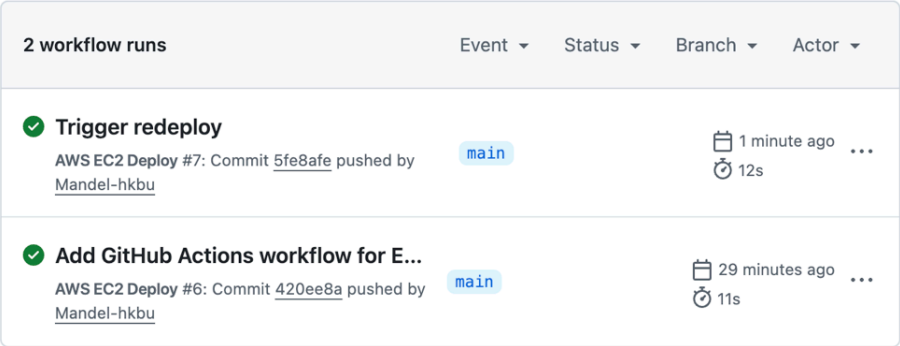
   - Test manual SSH login with your key pair.
2. Validate secrets:
   - Re-enter `EC2_HOST`, `EC2_USER`, and `EC2_KEY` if logs mention connection problems.
   - Confirm the SSH key matches the EC2 key pair used at instance creation.
3. Check workflow file:
   - Read and correct the content of deploy.yml in your local repository.
   - Check indentation and spacing in deploy.yml – GitHub Actions is sensitive to YAML formatting.
   - Run the following command if you modified deploy.yml:

     ```
     git add .github/workflows/deploy.yml
     ```

4. Redeploy safely:
   - Make a non-code commit to re-trigger deployment:

     ```
     git commit --allow-empty -m "Trigger redeploy"
     git push origin main
     ```

5. Monitor logs:
   - Open the failed workflow run, expand each step, and identify the exact error.
   - Correct the issue locally, commit, and push again.



Verification

- Once deployment succeeds, your chatbot runs on the EC2 instance.
- Confirm that the chatbot responds as expected.
- Verify the python process for your chatbot running in the EC2 instance by using the following command:

```
ssh -i "keypair_chatbot.pem" ec2-user@ec2-abc123.ap-east-1.compute.amazonaws.com ps -ef|grep python
```

> Replace `ec2-abc123.ap-east-1.compute.amazonaws.com` with the actual DNS name of your EC2 instance.

> You should properly see the following output:
> `ec2-user 37271 1 0 06:02 ? 00:00:00 /home/ec2-user/chatbot_comp7940/venv/bin/python chatbot.py`

Write-Up Questions

Submit answers to Moodle:

1. Identify the command used in `deploy.yml` that uploads/deploys your chatbot.
   → `ssh ...`
2. Draw a diagram showing connections and data flow between local repo, GitHub repo, and AWS EC2

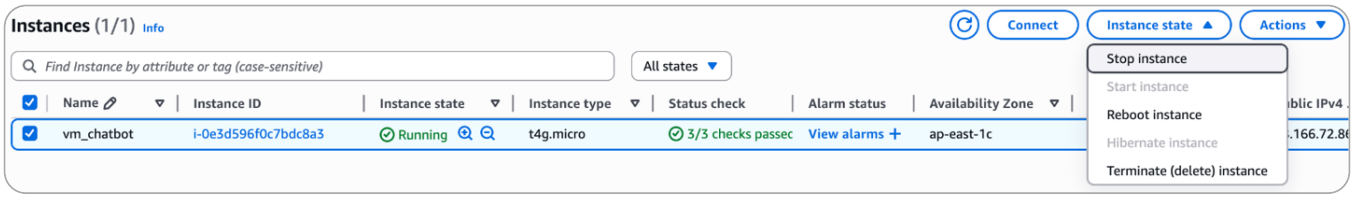
instance.

3. Explain why storing passwords in `config.ini` is insecure, and how GitHub Secrets improve security.
4. Provide screenshots of your GitHub Actions run and explain them.

Important: Shutdown your EC2 instance

Once you complete this lab, you should shut down your EC2 instance immediately to avoid unnecessary charge. Amazon offers Free Tier services for 6 months. You may be charged if you keep your EC2 instance up and running.

To shutdown your EC2 instance, select it from the instance list and then click `Instance state` > `Stop instance`.



If it is no longer needed, you should delete it by using `Terminate (delete) instance`.